



IOT ATTACK DETECTION THROUGH TCP ♦ MQTT

แชร์วิธีคิดจากข้อมูลจริง

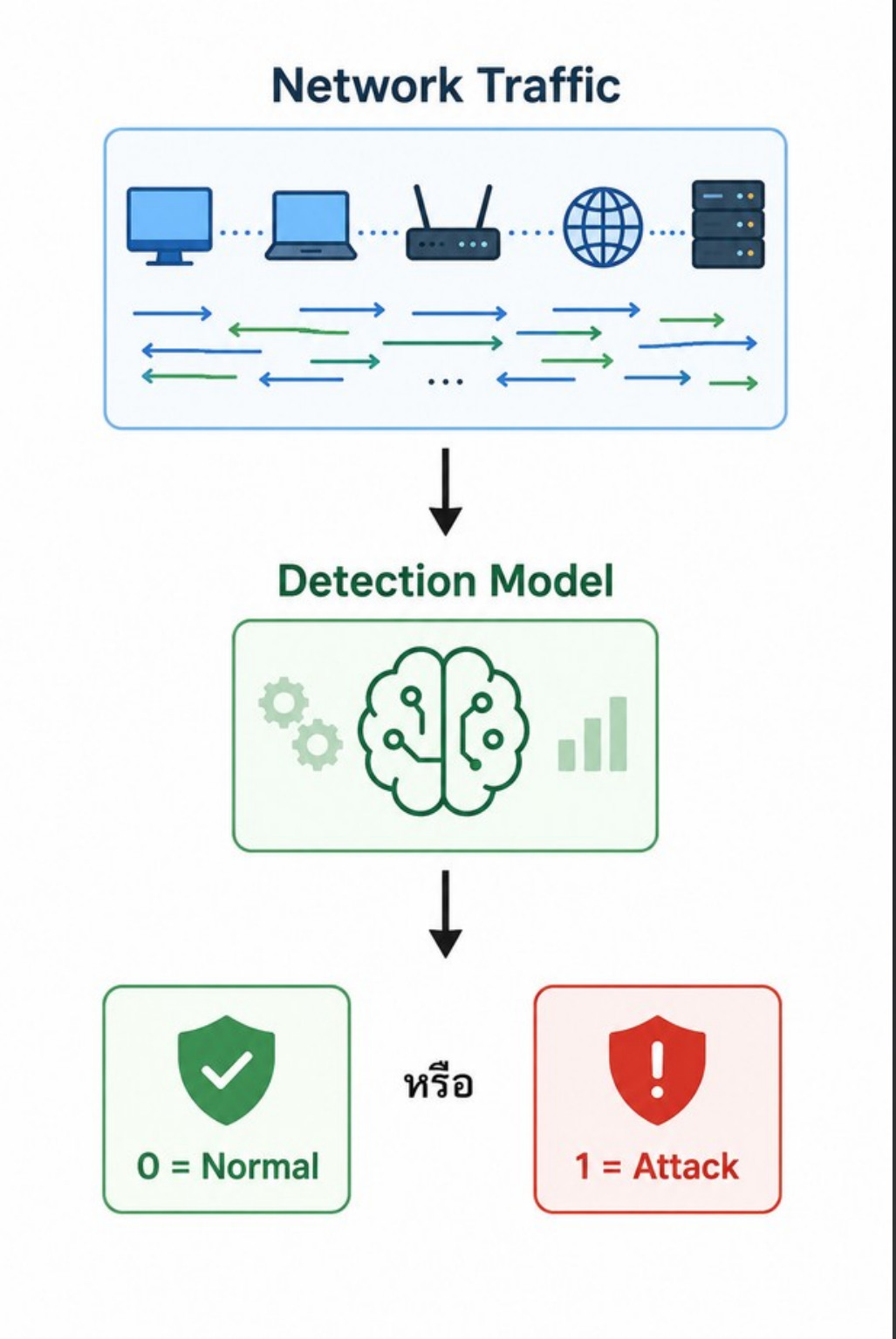
THE PROBLEM IN ONE SENTENCE

โจทย์จริง

เราต้องดู Network Traffic 10,000 แกว
แล้วตอบเพียง 2 ค่า

0 = Normal
1 = Attack

แต่ Train 100,000 แกว
มี Normal เท่านั้น



สิ่งที่ทำให้โจทย์นี้ยากคือเราไม่เคยเห็นตัวอย่าง Attack ใน Train เลยครับ เราจึงไม่สามารถสอนโมเดลแบบปกติว่า Attack หน้าตาเป็นอย่างไรได้ ต้องเริ่มจากการทำความเข้าใจว่า Normal มีรูปแบบแบบไหนก่อน

DATA CHALLENGE

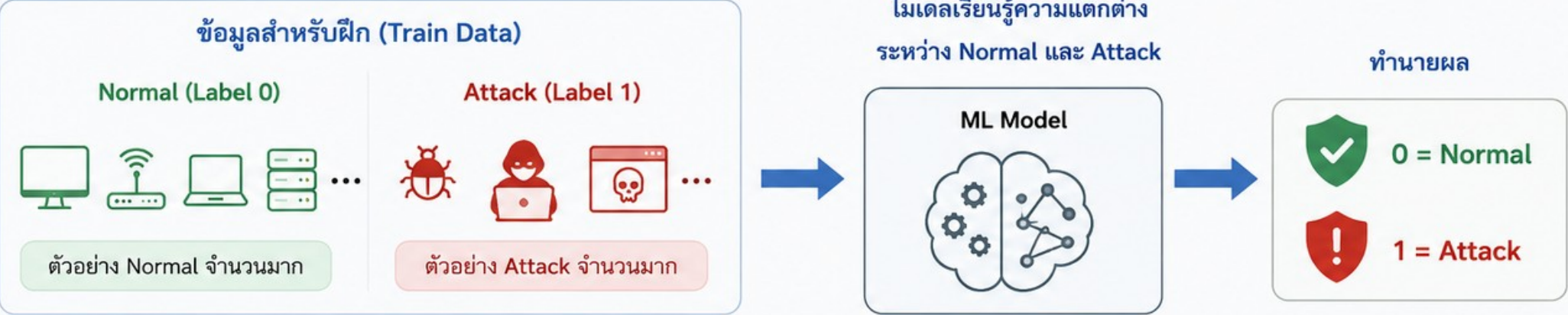
ข้อมูลที่ให้มา

TRAIN 100,000 rows
NORMAL ONLY

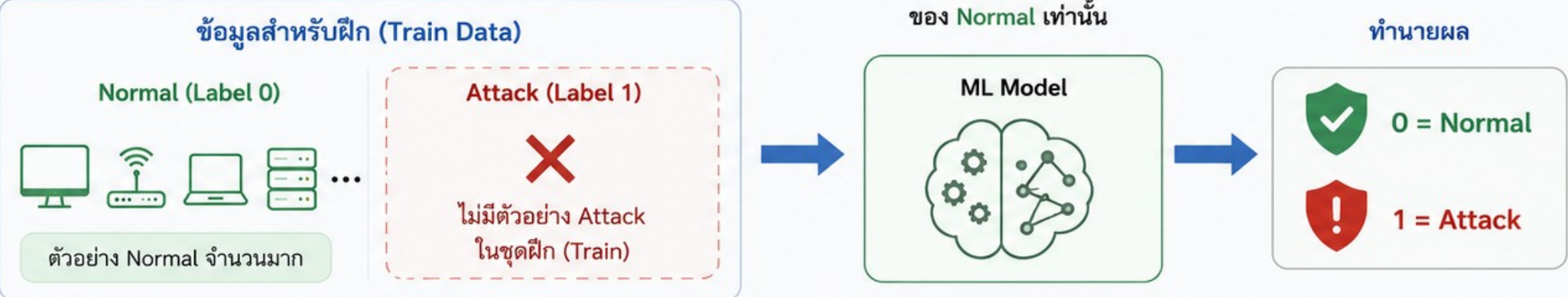
TEST 10,000 rows
NORMAL + ATTACK

22 network features + Id
TCP / MQTT / timing

ปกติการทำ Binary Classification



โจทย์นี้ (มีเฉพาะ Normal ใน Train)



⚠️ เนื่องจากโมเดลไม่เคยเห็นตัวอย่าง Attack ในการฝึก จึงไม่สามารถเรียนรู้ลักษณะของ Attack ได้โดยตรง ทำให้การตรวจจับขึ้นอยู่กับการที่ “ข้อมูลที่ไม่ใช่ Normal” จะถูกมองว่าเป็น 1 (Attack) หรือไม่



สรุป
ปกติ: ต้องมีตัวอย่างทั้ง 2 กลุ่ม (0 และ 1) เพื่อให้โมเดลเรียนรู้ความแตกต่าง
โจทย์นี้: มีเฉพาะ Normal ในการฝึก → ไม่สามารถใช้ Binary Classification แบบทั่วไปได้ง่าย ๆ

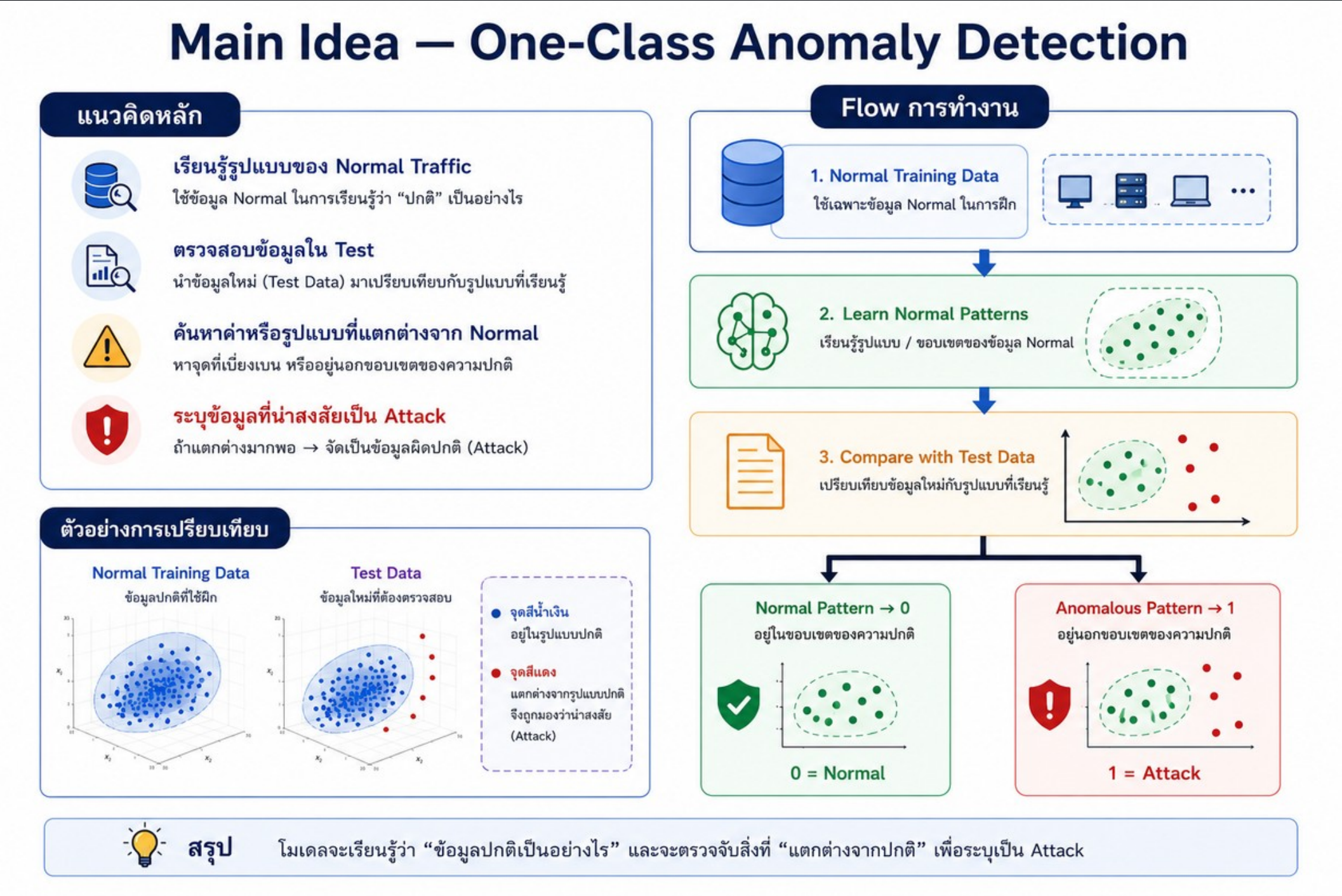
ต้องใช้เทคนิคพิเศษ
เช่น Anomaly Detection
หรือ One-Class Classification

Train มีแต่ Normal ทำให้โจทย์นี้เป็น One-Class Anomaly Detection มากกว่า Binary Classification ครับ จุดสำคัญคือข้อมูลหนึ่งแถวไม่ควรถูกตัดสินจากคอลัมน์เดียว แต่ต้องดูความสัมพันธ์ของ TCP, MQTT และบริบทของ connection

MAIN IDEA

ONE-CLASS DETECTION

- 1 เรียนรู้ Normal profile
- 2 เปรียบเทียบกับ Test
- 3 อ่านความหมาย TCP + MQTT
- 4 แจ้งเฉพาะความผิดปกติที่มีหลักฐาน



ผมไม่ได้ให้โมเดลจำว่า Attack ต้องมีหน้าตาแบบเดียว แต่ให้มันรู้จักขอบเขตของ Normal ก่อน จากนั้นจึงค่อยถามว่าแฉนี้แตกต่างเพราะเป็น Attack จริง หรือแค่เป็น Normal รูปแบบใหม่

เปรียบเหมือน รปภ. ที่รู้จักพฤติกรรมปกติของคนในอาคาร แม้ไม่เคยเห็น
ผู้บุกรุกมาก่อน ก็ยังสังเกตพฤติกรรมที่แตกต่างได้

FEATURES THAT MATTER

- TCP
- frame.len / tcp.len
 - tcp.window_size
 - SYN / ACK / RST
 - tcp.stream

- MQTT
- mqtt.msgtype
 - mqtt.kalive
 - mqtt.len / topic_len

1) Feature ที่สำคัญ

Feature	ความหมาย
 frame.len	ขนาดทั้งหมดของแพ็กเก็ต (รวมทุกส่วน: Ethernet + IP + TCP + Payload)
 tcp.len	ขนาดข้อมูลใน TCP Payload (เฉพาะข้อมูลที่ส่งจริง)
 tcp.window_size	ปริมาณข้อมูลที่ฝั่งรับพร้อมรับ (Receive Window)
 tcp.flags.syn	ใช้เริ่มต้นการเชื่อมต่อ TCP (Synchronize)
 tcp.flags.ack	ยืนยันรับการรับข้อมูล (Acknowledge)
 mqtt.msgtype	ประเภทข้อความ MQTT (เช่น CONNECT, PUBLISH, SUBSCRIBE เป็นต้น)
 mqtt.len	ขนาดข้อความ MQTT (รวม Header + Payload)
 mqtt.topic_len	ความยาว Topic ของ MQTT (จำนวนตัวอักษรใน Topic)

3) คอลัมน์ที่ไม่ใช้ตัดสินโดยตรง

 Id
เป็นเพียงรหัสของ แต่ละรายการ
เช่น 1, 2, 3, ...
ไม่มีผลต่อพฤติกรรมของแพ็กเก็ต

 tcp.stream
ใช้ระบุ Connection ภายใน Capture

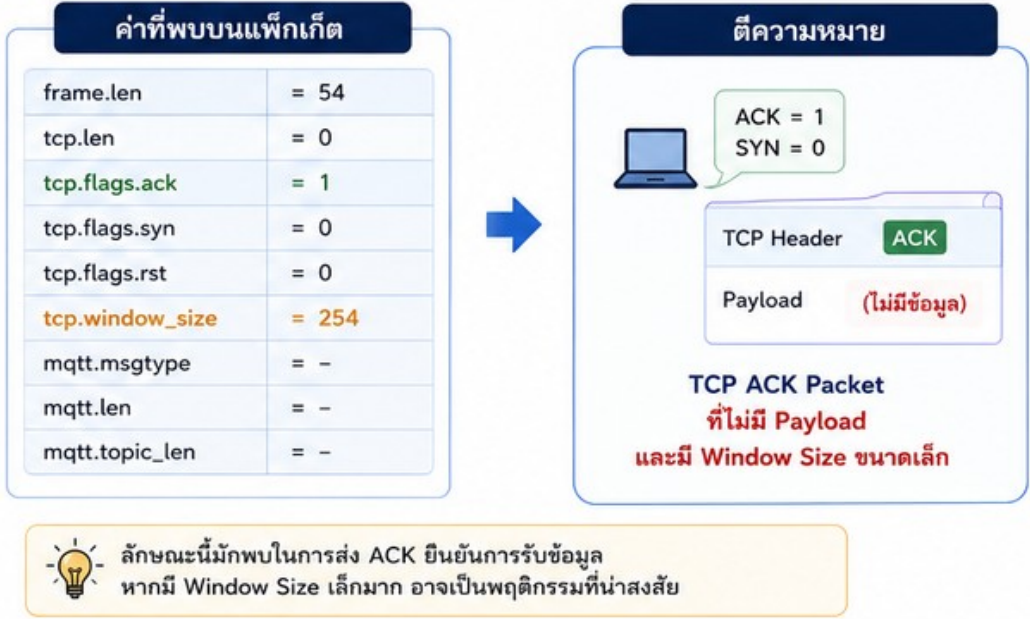
Stream = 5
ช่วยจัดกลุ่มแพ็กเก็ตใน Connection เดียวกัน

 tcp.analysis.initial_rtt
ค่า RTT เริ่มต้นที่วัดได้จากการเชื่อมต่อ

เปลี่ยนแปลงตาม Connection และสภาพเครือข่าย

คอลัมน์เหล่านี้ “ไม่ใช่” ในการตัดสินว่าเป็น Normal หรือ Attack โดยตรง แต่ใช้เป็นข้อมูลประกอบในการวิเคราะห์เท่านั้น

2) ตัวอย่างรูปแบบแพ็กเก็ต



4) เปรียบเทียบตัวอย่าง: Normal vs Anomalous

ตัวอย่าง Normal
เช่น การสื่อสารทั่วไป

frame.len	≈ 100 – 1500
tcp.len	> 0 (มี Payload)
tcp.flags.ack	1
tcp.flags.syn	0 หรือ 1
tcp.flags.rst	0
tcp.window_size	ขนาดปกติ (เช่น 1024 – 65535)
mqtt.msgtype	มีค่า
mqtt.len	> 0
mqtt.topic_len	> 0

ตัวอย่างที่น่าสงสัย (Anomalous)
เช่น อาจเป็นการโจมตีบางรูปแบบ

frame.len	≈ 54
tcp.len	= 0
tcp.flags.ack	1
tcp.flags.syn	0
tcp.flags.rst	0
tcp.window_size	เล็กมาก (เช่น 251 – 255)
mqtt.msgtype	มักไม่มี
mqtt.len	= 0
mqtt.topic_len	= 0

สรุปแนวคิด

เราไม่ได้ดูแค่ค่าหนึ่งค่า แต่ดูความสัมพันธ์ของหลาย Feature ร่วมกัน

↓

หากรูปแบบใดแตกต่างจากสิ่งที่ Normal เคยเรียนรู้ไว้มาก จะถูกจัดเป็น

Attack (1)

หากใกล้เคียงปกติ จะจัดเป็น

Normal (0)



สรุป: โมเดลจะเรียนรู้ “รูปแบบของ Normal” จากข้อมูลฝึก แล้วตรวจสอบข้อมูลใหม่ (Test) หากรูปแบบแตกต่างจากปกติมาก → น่าสงสัย → ทำนายเป็น Attack (1)

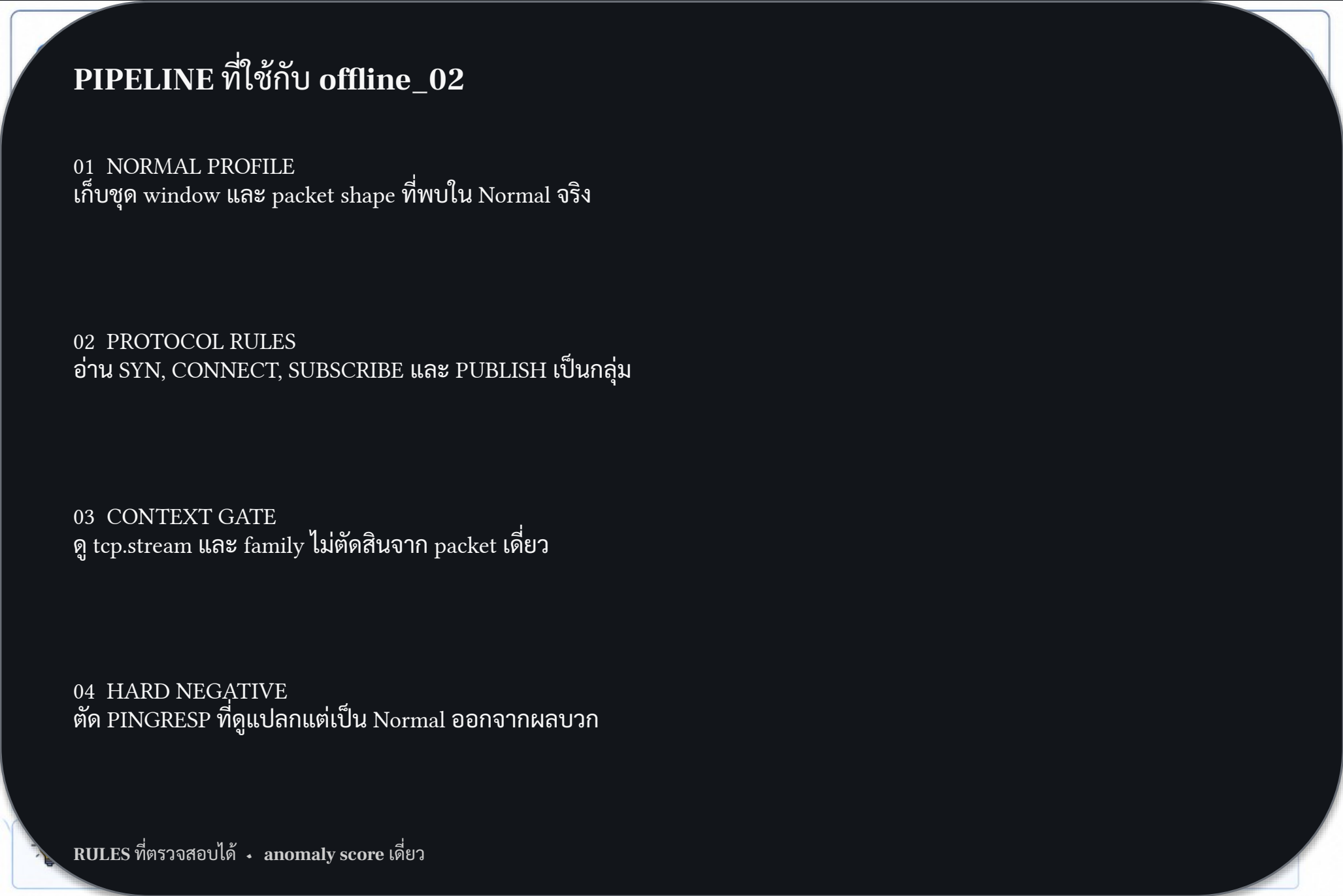
● 0 = Normal

● 1 = Attack

Feature ที่ผมให้ความสำคัญมีสองกลุ่มครับ กลุ่มแรกคือ TCP ซึ่งบอกขนาด packet และ state ของ connection กลุ่มที่สองคือ MQTT ซึ่งบอกว่า packet นี้เป็น CONNECT, PUBLISH หรือ SUBSCRIBE การอ่านสองกลุ่มร่วมกันช่วยลดการตีความผิดพลาดจากค่าที่ดูแปลกเพียงตัวเดียว

FROM RULES TO A ROBUST MODEL

- 1 Normal profile
set ของ window และ packet shape ที่พบจริง
- 2 Protocol rules
SYN flood / CONNECT / SUBSCRIBE / PUBLISH
- 3 Context gate
ดู stream และ family ไม่ดู packet เดี่ยว
- 4 Hard negative
ตัด PINGRESP ที่ดูแปลกแต่เป็น Normal

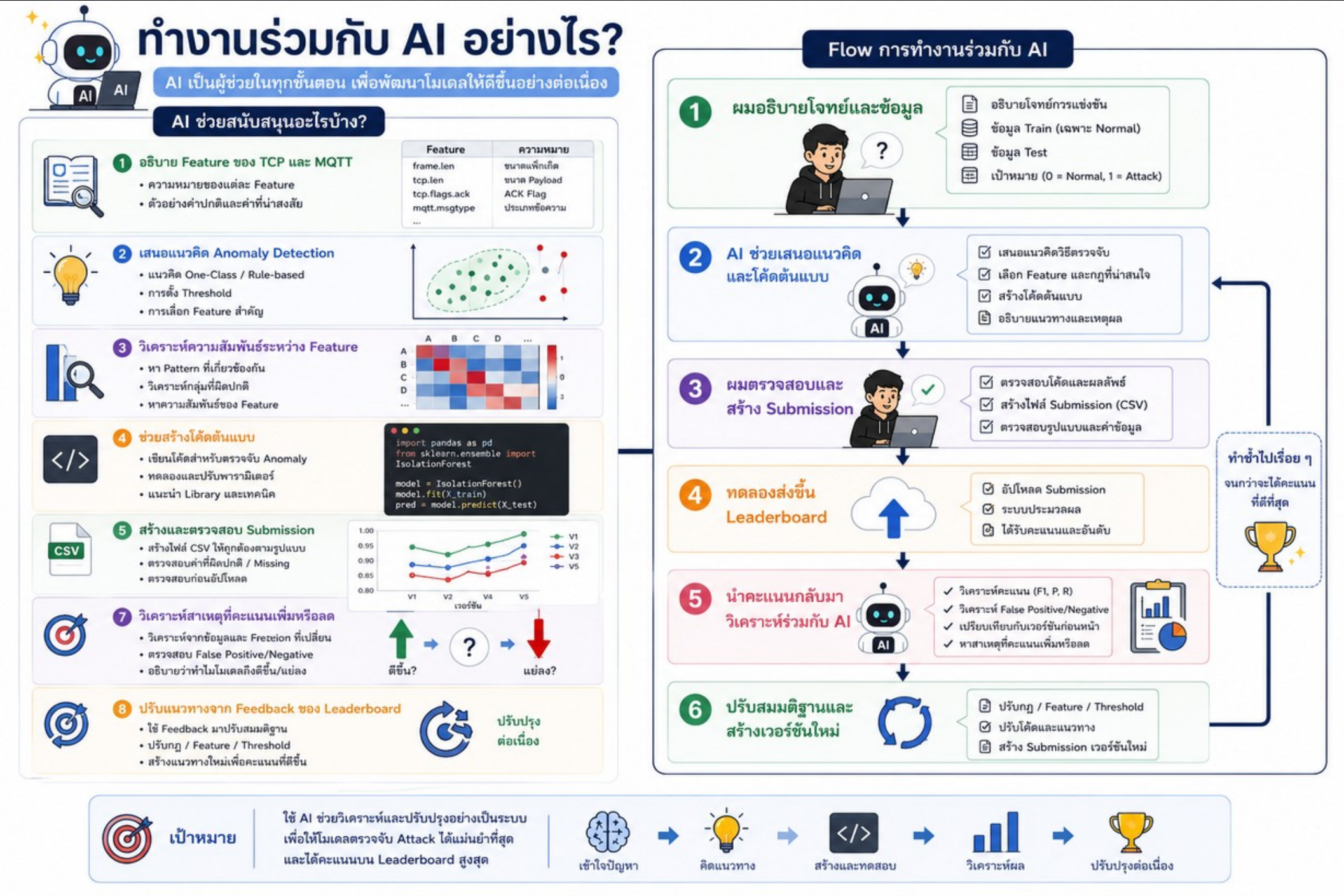


ผมเริ่มจากกฎที่อธิบายได้ก่อนครับ เพราะเรายังไม่มี Attack label ที่เชื่อถือได้ กฎไม่ได้ใช้ anomaly score เดี่ยว แต่ใช้หลาย field ประกอบกัน
แล้วมี hard-negative filter คอยดึง false positive ออก

AI + HUMAN VALIDATION

- AI ช่วย
- อธิบาย feature
 - เสนอ rule และ code
 - จัดอันดับ residual rows
 - ทดลอง ExtraTrees / RandomForest / HistGradientBoosting / IsolationForest

- คนตัดสินใจ
- อ่าน protocol
 - เช็ค stream context
 - หักล้าง false positive



AI ช่วยให้เราตั้งสมมติฐานและทดลองได้เร็วขึ้น แต่ผมไม่ใช่ผลจาก AI ครั้งเดียวแล้วเชื่อกันที่ครับ ทุกแนวคิดต้องกลับมาตรวจด้วย protocol meaning, stream context และ hard negatives โดยเฉพาะ ML models ที่ใช้ pseudo-label เพราะมีความเสี่ยงเรื่อง circularity

THE RESULT WE EXPLAIN

submission_offline_02_
hedge_payload132.csv

ROWS 10,000
POSITIVE LABELS 2,811
PRIVATE F1 0.97198
PUBLIC F1 0.96267

BASELINE + STRUCTURAL Id=9816
+ PAYLOAD132 HEDGE Id=1145

SELECTED FILE | offline_02

FILE
submission_offline_02_hedge_payload132.csv

SCORE
PRIVATE F1 0.97198
PUBLIC F1 0.96267

DATA
10,000 rows | 2,811 positive labels

CHANGE
baseline + structural Id=9816
+ payload hedge Id=1145

ตัวเลขเป็นคะแนนที่ผู้ใช้รายงานจาก Kaggle

ไฟล์ที่เลือกสำหรับอธิบาย

ไฟล์ที่ผมนำมาอธิบายในวันนี้คือ offline_02 ครับ ไฟล์นี้ต่อยอดจาก baseline ด้วย structural row ที่ช่วยเติมรูปแบบ SYN stream และ payload hedge ที่ Id 1145 โดยตั้งใจเพิ่มเฉพาะกลุ่มที่มีเหตุผล ไม่ได้กวาดทุกแถวที่ดูผิดปกติ

ผลที่รายงานคือ Private F1 0.97198 และ Public F1 0.96267

- 1 Rare ไม่ได้แปลว่า Attack
- 2 Context สำคัญกว่า packet เดียว
- 3 ทำนาย Attack มากไป F1 อาจลด
- 4 ML score ไม่ใช่หลักฐาน
- 5 กล้วยๆ แต่ตรวจสอบได้

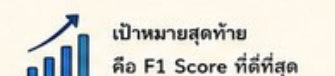
4 แนวคิดสำคัญในการทำ Anomaly Detection ✨

💡 ดังนั้น Anomaly จึงเป็น “ผู้ต้องสงสัย” ไม่ใช่ “จำเลย” เสมอไป

⚠ ทำนาย Attack มากเกินไป → False Positive เพิ่มขึ้น → Precision ลดลง → F1 Score ลดลง

★ เป้าหมายคือ “สมดุล” เพื่อให้ F1 Score สูงสุด

 ทดลอง → วัดผล → เรียนรู้ → ปรับปรุง → ทำซ้ำ
นี่คือกฎแห่งการพัฒนาาระบบที่ได้ผลลัพธ์ที่ดีที่สุด



FINAL TAKEAWAY

Normal-only data



Learn protocol behavior



Detect meaningful deviations



Validate with evidence

ไฟล์ที่นำมาอธิบาย
offline_02
Private 0.97198
Public 0.96267

PROTOCOL-FIRST SUMMARY

01 NORMAL-ONLY
เรียนรู้ขอบเขตจาก Train ที่มี Normal เท่านั้น

02 TCP + MQTT
ใช้ feature ที่มีความหมายของ protocol ไม่ใช่ค่าที่แปลกอย่างเดียว

03 CONTEXT + NEGATIVE
ดู stream และหักล้าง false positive ก่อนตัดสิน

04 RESULT
offline_02 | Private F1 0.97198 | Public F1 0.96267

เรียนรู้ --> ตรวจสอบหลักฐาน --> เลือกเฉพาะ deviation ที่มีเหตุผล

บทสรุป

สรุปคือผมเปลี่ยนข้อจำกัดของข้อมูล Normal-only ให้เป็นวิธีคิดแบบ protocol-first ครับ เริ่มจากเรียนรู้พฤติกรรมปกติ ใช้ TCP และ MQTT เป็นหลักฐาน เพิ่ม context ระดับ stream และใช้ hard negatives ป้องกัน false positive นี่คือเหตุผลที่ไฟล์ offline_02 อธิบายได้ทั้งวิธีทำและเหตุผลของผลลัพธ์

THANK YOU

**FOR LISTENING
QUESTIONS?**